

MCP SERVER · ALPACA · STOCKSHARP

Odysseus

Turns any AI agent into a quantitative researcher — and holds it to the discipline that makes a result mean something.

THE PROBLEM

An agent asked to find a profitable strategy will find one.

Give a capable model market data and a backtester and it will hand you an equity curve within the hour. It will also have searched the same data it is reporting on. The only thing that separates a finding from a search is data the search could not reach.

WHAT IT IS

An MCP server. Nothing else.

No model inside it, no chat, no web application. You run it beside **your own agent** and, for live data, **your own Alpaca key**. Your agent brings the reasoning; this brings the parts an agent is bad at alone.

Data that cannot move

History is downloaded once and frozen under a content hash. Every number traces back to exactly those bars.

A real strategy

The hypothesis is translated deterministically into a StockSharp **Strategy** class, compiled by Roslyn, and run on the same engine as a live one.

Data it may not see

A slice held back from the agent for the whole of the research, measured once, and spent across projects rather than per project.

THE LOOP

What the agent actually calls

<code>import_history</code>	real bars, frozen and hashed
<code>analyze_market</code>	what the instrument does, before anything is proposed for it
<code>propose_spec</code>	the hypothesis, as a formal document the server checks
<code>build_candidate</code>	translated to C#, compiled, hashed
<code>run_optimization</code>	a seeded search over the numbers the specification declared
<code>evaluate_candidate</code>	six fixed runs, reported in full
<code>measure_on_closed_data</code>	once, against data nobody was allowed to see
<code>complete_strategy</code>	you decide; the server keeps the code and every number behind it

THE PRINCIPLE

It measures. It does not judge.

There is no score in this server, no threshold, no accept-or-reject. Whether a drawdown is bearable or a return is worth having depends on what is being researched — and a threshold compiled into a server is one person's opinion, frozen, arriving dressed as arithmetic. The one conclusion in the product comes from outside it, through `complete_strategy`, and the server's part is to write down what was concluded and everything needed to check it later.

Three slices, and one of them is not for looking at

55–65%

Development

Form the hypothesis here. Search the parameters here. This is the part the agent is allowed to fit.

15–25%

Validation

Measured on, never searched on.

the rest

Closed

Boundaries never disclosed.
Measured exactly once.

The proportions come from the dataset's own content hash, so the boundary cannot be guessed from the product. And the closed slice is spent once per stretch of history, recorded in a ledger beside the projects — because starting a second project and importing the same dates does not make the answer unseen.

WHAT A MEASUREMENT IS

Six runs, always the same six

Which runs those are is not the caller's to choose. An agent that could pick its own evidence would pick the evidence that suits it.

baseline

Development, then validation. Fees and the spread charged separately, because only one of them can be traded around.

costs $\times 1.5$

The same held-out slice with costs half again as high. What survives is the part that did not depend on the spread being what was assumed.

entry one bar later

The order placed a candle after the signal. A result that survives one and not this was resting on being fast rather than on being right.

Plus three consecutive walk-forward windows, which ask a different question again: whether one stretch of market carried the whole result.

Zero trades and no edge are the same row of zeros

They are opposite findings. One says the hypothesis did not describe anything that happened on that slice; the other says it did, and the market paid nothing for it.

```
held out           : net 0.00   trades 0
held out, +50% costs : net 0.00   trades 0
```

```
why nothing happened : [heldOut] The rules never triggered on this slice: no
                        order was placed at all. That is an answer about the
                        hypothesis rather than a failure – the condition as
                        written did not occur here. Check the warm-up is not
                        eating the slice, then loosen the condition or test it
                        where the market does what it describes.
```

A run with no trades reports the same empty numbers whether the rules never fired or the orders were too large for the bars to fill. **Every empty run carries which of the two it was.**

NOT A CONFIGURATION FILE

The hypothesis becomes real code

```
// entries.e1: opens Long
if (Position == 0 && ((candle.ClosePrice > _seriesHighest.At(1))
    && (candle.TotalVolume > (v_volumeSma * VolumeMultiplier))
    && (MarketTime(candle.OpenTime) >= new TimeSpan(9, 50, 0))))
{
    LogInfo("Entry rule e1 of the specification held.");
    BuyMarket(Volume);
    return;
}
```

An ordinary StockSharp strategy: `OnStarted2`, a candle subscription, orders placed with `BuyMarket`. The translation is deterministic — the same specification produces the same source, byte for byte, and the hash of that source is what makes two candidates the same candidate. **What research finished with is a file you can open, read and run.**

Measure the instrument before proposing anything for it

```
Odysseus.Server.exe

--> {"method": "tools/call", "name": "analyze_market", "arguments": {"projectId": "prj_5fc5be33...", "symbol": "NVDA"}}
<-- {
  "coverage": {
    "symbol": "NVDA",
    "bars": 6720,
    "sessions": 35,
    "from": "2026-05-28T08:00:00Z",
    "to": "2026-07-17T23:55:00Z",
    "medianVolume": 91985,
    "highVolumeBarsPercent": 13.2436
  },
  "movement": {
    "medianBarRangePercent": 0.1668,
    "upperDecileBarRangePercent": 0.4465,
    "medianDailyRangePercent": 3.6426,
    "annualisedVolatilityPercent": 41.4751
  }
}
(0.2 s)
```

What it was charged, and the account through the run



What it will not do

- It will not tell you a strategy is **good**. It reports what happened and leaves the reading to you.
- It will not trade **real money**. Paper accounts only, and never an order the strategy did not place.
- It will not measure the **closed slice twice**. That refusal is the whole of its value.
- It will not pretend a **backtest is a result**. The emulator fills against bars; a fill that never comes is the most common difference between a backtest and an account.

37

MCP tools, each describing itself to the agent that has to choose it

384

tests, including the whole loop over a real stdio transport

MIT

our code; StockSharp arrives as NuGet under its own terms